

OOPJ Day 5

Fri, 11 Sept 26

Date Class Logic: Diagnosing Common Mistakes

- Root cause of student struggles: logic gaps, not Java syntax
 - Every student can write for loops, while loops, switch cases, if-else chains
 - Java is not the blocker; the same assignment in C/C++ would produce identical confusion
 - Logic improves only with practice; overnight mastery is not expected
- Abhay's code: detailed walkthrough of bugs
 - Month validation skipped Aug (8), Oct (10), Dec (12) in the day-range check
 - maxDay hardcoded per month inside the while loop but never recalculated after month++
 - Example: date 15/9/2026 + 100 days → day becomes 115; loop subtracts 30 (Sept), month becomes 10; next iteration subtracts 31 (Oct), month becomes 11; subtracts 30 (Nov), month becomes 12; 24 days remain, within range, date set to 24/12/2026
 - Leap year check applied unconditionally regardless of month
 - Should only trigger when month == 2; entering the else-if for any other month is a logic error
 - Month-13 rollover placed in an else-if instead of a standalone if
 - Causes an unnecessary extra loop iteration before year increments
 - Fix: make the year-rollover check a separate if so it fires immediately when month > 12
- Mukul's observation: the while loop always subtracted a fixed 30 days regardless of actual month length
 - maxDay must be recalculated every time month++ executes inside the loop

Om's Approach: Helper Function Design

- Om used a daysInMonth(month) helper method instead of repeated if-else chains
 - Passes current month; returns exact day count (28, 29, 30, or 31)
 - Leap year logic lives inside this one method; any future fix propagates automatically
 - Abhay's approach duplicated the same if-else in setDate, addDays, addMonths, addYears: four places to fix if a bug is found
- Performance tradeoff in Om's addDays

- Om's for loop increments one day at a time: adding 31 days runs the loop 31 times
- Abhay's bulk subtraction approach is far faster: each iteration removes a full month's worth of days
- Instructor verdict: Om's logic is correct; performance is poor but acceptable at the learning stage
- Priority order: get output first → review code → optimize performance → optimize memory
- Code size benefit
 - Om's version is significantly shorter and easier to maintain
 - Fewer lines = lower development time, lower maintenance time, fewer places for bugs to hide

Optimized Date Class Design

- Replace all if-else month chains with a private int array
 - Declare: `private int[] numberOfDays = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}`
 - 13 entries; index 0 is a dummy value because months start at 1
 - Direct index lookup: `numberOfDays[month]` returns the correct day count instantly
 - Old approach: checking `month == 1, 3, 5, 7, 8, 10, 12` one by one for month 12 requires 7 comparisons; array jumps directly to index 12 in one CPU operation
 - Memory cost: 13 integers = ~52 bytes; negligible on any modern machine
- `isLeapYear()` as a reusable private boolean method
 - Conditions: `year % 4 == 0 AND (year % 400 == 0 OR year % 100 != 0)`
 - Single condition is not sufficient; both divisibility rules must be checked
 - Call this method whenever the month changes to February; update `numberOfDays[2]` to 29 or 28 accordingly
- `setDate` reduced to a few lines
 - Year validation: if `year < 1950` or `year > 3000`, default to 2026
 - Month validation: if `mm < 1` or `mm > 12`, default to 1
 - Day validation: if `dd < 1` or `dd > numberOfDays[month]`, default to 1
 - Leap year adjustment applied before day validation so February limit is always correct
- Parameterized constructor to enforce valid object creation
 - Default constructor leaves day/month/year as 0/0/0 (invalid state)
 - Define a constructor: `Date(int dd, int mm, int yy)` that calls `setDate` internally
 - Once a custom constructor is defined, the compiler removes the default constructor
 - Programmer is forced to supply all three values; object cannot be created in an invalid state
 - Demonstrated live: passing invalid values still produces a valid date because `setDate` validates them

addDays as the Single Source of Truth

- All date arithmetic funnels through addDays; no other method duplicates the logic
- addDays algorithm:
 - `totalDays = day + daysToAdd`
 - While `totalDays > numberOfDays[month]`: subtract `numberOfDays[month]`, increment month
 - If month > 12: reset month to 1, increment year, recalculate `isLeapYear`
 - `day = totalDays` (remainder after all subtractions)
- Live trace: 15/9/2026 + 100 days
 - `totalDays = 115`; Sept has 30 → 85 remain, month = 10
 - `85 > 31` (Oct) → 54 remain, month = 11
 - `54 > 30` (Nov) → 24 remain, month = 12
 - `24 ≤ 31` (Dec) → day = 24; result: 24/12/2026
- addMonths design
 - Loop from temp = 0 to monthsToAdd: accumulate `numberOfDays[currentMonth + temp]` into `inDays`
 - Call `addDays(inDays)`; all validation (leap year, month overflow, year rollover) handled there
 - Avoids hardcoding 30 days per month
- addYears design
 - `monthsToAdd = yearsToAdd × 12`
 - Call `addMonths(monthsToAdd)`; cascades down to `addDays` automatically
- Live trace: 31/1/2026 + 1 month
 - Jan has 31 days → `addDays(31)` → `totalDays = 62`
 - `62 > 31` (Jan) → 31 remain, month = 2; not leap year → `numberOfDays[2] = 28`
 - `31 > 28` (Feb) → 3 remain, month = 3
 - `3 ≤ 31` (Mar) → day = 3; result: 3/3/2026
- Live trace: 29/1/2024 + 1 month (leap year edge case)
 - Jan has 31 days → `addDays(31)` → `totalDays = 60`
 - `60 > 31` (Jan) → 29 remain, month = 2; 2024 is a leap year → `numberOfDays[2] = 29`
 - `29 == 29` → not greater → day = 29; result: 29/2/2024 ✓
 - Same date in 2023 (non-leap): `29 > 28` → 1 remains, month = 3; result: 1/3/2023 ✓

Code Quality and Naming Conventions

- Class names must reflect real-world entities
 - Date not Date1, CalDate, or CalculateDate
 - Rename via IDE: right-click → Refactor → Rename; updates all references automatically
 - Keeping a broken class around as “DateOld” is acceptable temporarily; delete or rename promptly

- Method names must express intent clearly
 - addMonths not setMonth (setter implies replacement; adder implies accumulation)
 - addDays not newDD; daysToAdd not mm or i
 - Readable names matter when the class is used as a library by other programmers
- Variable naming discipline
 - i is banned as a loop variable; use temp, counter, or a descriptive name
 - Object references: objCalculator is clearer than cal
 - In professional environments, every variable must be documented: purpose, scope, lifetime
- Curly brace discipline
 - Single-statement if blocks do not need curly braces
 - Unnecessary braces create a stack frame, allocate and deallocate memory, and inflate code size
 - Example: if (mm < 1 || mm > 12) month = 1; else month = mm; — no braces needed
- No console I/O inside Date class
 - No System.out.println or System.in.read anywhere in Date
 - Keeps the class usable on console, GUI, mobile, and web applications without modification
 - The Date class is a library; the program class handles all I/O

Inheritance: Concepts, Types, and Java Syntax

- Definition: the ability to create a new class by extending an existing one, adding or customizing properties and behaviors
 - Customization is the key motivation; if no customization is needed, inheritance is unnecessary
- Java keyword: extends
 - class Derived extends Base (replaces C++ colon-public syntax)
 - Base is the superclass; Derived is the subclass
- Four types of inheritance:
 - Single: one parent → one child (A → B)
 - Multi-level: A → B → C (grandfather → father → child); each level adds features
 - Multiple: C inherits from both A and B simultaneously; C has members of A, B, and its own
 - Hybrid: combination of multi-level and multiple (A → B, A → C, then B + C → D)
- Multiple inheritance for classes is not supported in Java
 - Reason: ambiguity when two parent classes share a data member or method name
 - Example: A has num1; B has num1; C inherits both → obj.num1 is ambiguous
 - Same ambiguity applies to methods: obj.display() — which parent's display runs?

- Analogy: four students named Mukul in a class; calling “Mukul, leave” is ambiguous
- Java solution: multiple inheritance is supported via interfaces (covered later)
- Object of a derived class in memory
 - Contains the full base section plus the derived section
 - Example: Base has n1, n2, n3 (12 bytes); Derived adds n4, n5 (8 bytes); derived object = 20 bytes total
 - A base class reference can point to a derived object because the base section is always present within it

Upcasting and Downcasting

- Upcasting: assigning a derived object to a base class reference
 - Implicit; no explicit cast operator required
 - Valid because the derived object always contains the complete base section
 - Compiler knows the derived class contains the base; compatibility is guaranteed at compile time
 - Going up the hierarchy: Derived → Base (Man → Human → Ape)
 - Example: Base obj = dObj; — no cast needed; obj can access n1, n2, n3 and display only
 - Printing obj shows the derived class name and hashCode because the underlying object is still of the derived type
- Downcasting: assigning a base reference back to a derived reference
 - Always explicit; programmer must supply the cast operator: Derived xObj = (Derived) obj;
 - Necessary when specific derived members (e.g., n4, n5, show()) need to be accessed again
 - Compiler cannot verify safety at compile time: memory allocation happens at runtime, so the compiler only knows the reference type, not what it actually points to
 - JVM performs a second type check at runtime
 - If the underlying object is genuinely of the derived type: cast succeeds
 - If the underlying object is only of the base type: JVM throws `ClassCastException`
 - C++ contrast: type checking only at compile time; invalid downcast causes crash or garbage values; Java prevents this at runtime
- Practical scenario demonstrating why explicit downcast is needed
 - Base obj = null; if (choice == 1) obj = new Base; else obj = new Derived;
 - Compiler cannot evaluate the if condition at compile time (memory allocation is a runtime event)
 - Compiler only knows obj is of type Base; it cannot know whether it points to Base or Derived
 - Programmer must explicitly cast and accept responsibility; JVM validates at runtime

Type Checking, Casting, and Conversion

- Three distinct concepts (commonly confused):
 - Type checking: compiler verifies that RHS type is compatible with LHS type before assignment
 - Principle: the RHS value must be completely and correctly assignable to the LHS without data loss
 - Example: `float total = 1570.65; int sum = total;` → compiler error: possible loss of data
 - Typecasting: programmer overrides the compiler's type-checking warning
 - Syntax: `int sum = (int) total;`
 - Does not convert; assigns as-is with accepted data loss (1570.65 stored as 1570, decimal dropped)
 - Tells the compiler: "I know there is a problem; proceed anyway"
 - In C/C++: even completely incompatible types can be force-cast, resulting in garbage values
 - In Java: compiler allows the cast if there is an inheritance relationship; JVM re-validates at runtime
 - Type conversion: transforms the value to fit the target type correctly
 - Example: 1570.65 rounds to 1571; 1570.35 rounds to 1570
 - Produces a meaningful result rather than a truncated one
- Java as a strongly typed language
 - Type checking occurs at compile time (compiler) and at runtime (JVM)
 - A forceful typecast between unrelated classes (no inheritance link) is rejected by the compiler outright
 - A forceful typecast between related classes passes the compiler but may throw `ClassCastException` at runtime if the actual object type is incompatible

Exercises Assigned and Tomorrow's Plan

- Three exercise sets shared in chat today:
 - For loop exercises (~15 small programs; e.g., area and perimeter of a circle given radius)
 - Array exercises
 - Classes exercises
- Each exercise should take no more than 10-15 minutes
- Goal: build comfort and fluency with Java syntax; reinforce concepts covered so far
- Project structure suggestion: one project per topic; use separate classes or methods per exercise rather than creating a new project for each
- Date class assignment remains active and must not be ignored
 - Recreate from scratch using today's optimized design (array-based `numberOfDays`, centralized `addDays`)
 - Focus only on business logic: `setDate`, `addDays`, `addMonths`, `addYears`
 - No need to rebuild the menu-driven program

- Test with all permutation combinations including leap years, month-end rollovers, and year boundaries
- After output is confirmed correct, review and optimize further

Next Steps

- **Recreate the Date class from scratch using the optimized design** Use the array-based numberOfDays approach, isLeapYear() helper, parameterized constructor, and centralized addDays; test all edge cases before optimizing further.
- **Complete all three exercise sets (for loop, arrays, classes)** Shared in chat today; target completion before tomorrow's 9 AM session.
- **Install Visual Studio Community Edition (C++ component only)** At least one or two students should have it ready; needed for tomorrow's JVM internals and dynamic polymorphism demo in C++.

Chat with meeting transcript:

<https://notes.granola.ai/t/d6cb7458-dc62-453b-a836-1afaf5b0d95d-00demib2>